

**ANKARA
YILDIRIM BEYAZIT
UNIVERSITY**

ANKARA YILDIRIM BEYAZIT UNIVERSITY

**Faculty Of Aeronautics And Astronautics,
Department Of Aerospace Engineering, Aerospace
Engineering**

AEE-323

**ENGINEERING EXPERIMENTATION
AND DATA ANALYSIS**

PROJECT

Group - 3

Author: Mustafa ÜÇÖZ

Student number: 21110111006

Author: Duygu Selin GÜRKAN

Student number: 2011011013

Author: İdris Nebi UZUN

Student number: 2011011015

December 15, 2024

Abstract

In this project, we aim to apply low-pass filtering to the data provided in the “Group-Data_3.mat” format. The dataset consists of three key components: time (time vector), measurement (noisy signal), and original (noise-free reference signal). The main objective is to filter the measurement data using a Finite Impulse Response (FIR) low-pass filter and compare the resulting filtered signal with the original noise-free signal.

The low-pass filtering process involves the careful determination of critical filter parameters, including the filter order, transition width, and cutoff frequency. Low-pass filters are widely used to suppress high-frequency noise while preserving essential low-frequency components of a signal [1].

Throughout the study, various visualizations have been generated to illustrate each step of the filtering process, including time domain and frequency domain comparisons, the filter kernel structure, and its frequency response. Frequency analysis is performed to identify the characteristics of the signal and the effects of filtering, ensuring that high-frequency noise is suppressed while preserving the essential low-frequency components of the signal[2].

By comparing the filtered signal with the original data, the results demonstrate the effectiveness of low-pass filtering in reducing noise and improving signal quality. The filtered signal was compared with the original data. A comment was made about the reason for the error in the measurement data[3].

1. Nomenclature

- f_s : Sampling frequency
- FFT: Fast Fourier Transform
- IFFT: Inverse Fast Fourier Transform
- FIR: Finite Impulse Response

2. Introduction

The aim is to start by answering the questions of what filtering is, why it is done, and under what circumstances it is performed, providing a general understanding of filtering. Filtering ensures that the data obtained through specific measurements is transformed into the desired form. There are various filtering methods to achieve the desired data. These methods, of course, depend on what kind of data you want and what kind of data you have.

The subject of this report is to filter the noisy measurement data among the given datasets using the low-pass filtering method, compare it with the original data in the time domain, and provide information about filtering. A low-pass filter suppresses frequency values beyond a specific dominant frequency value, aiming to reduce the amount of noise in the noisy data[2, 8].

The specific frequency value is determined by analyzing the frequency domain of the given data. By examining the frequency domain of the data to be filtered, peak points are identified. The extent to which these peaks are higher compared to other points is investigated, and the determined frequency value is referred to as the cut-off frequency. Normally, all peak frequency values should be considered; however, the reasoning behind focusing on two peak values has been emphasized very carefully and meticulously in the later sections. This reasoning is fundamentally based on the availability of the original data itself.

Once the cut-off frequency is determined, filtering is performed using the FIR (Finite Impulse Response) filtering method[9, 10].. Important comments and figures regarding the order and transition width have been presented and sufficiently explained.

The filtered signal obtained from the filtering process has been compared with the original signal in the time domain, and the most accurate values for the cut-off frequency, order, and transition width have been specified.

3. Methodology

3.1 Data Preparation

The dataset (`GroupData_3.mat`) includes:

- `timevec`: Time values (seconds).
- `measurement`: Noisy signal.
- `original`: Noise-free reference signal.

Simulation parameters were derived as follows:

Data Preparation

The dataset used in this study, `GroupData_3.mat`, comprises time-domain measurements of a noisy signal and a noise-free reference signal. The sampling frequency (f_s) was calculated based on the time vector:

The sampling frequency (f_s) is defined as the number of samples taken per unit time (usually per second) to convert a continuous time signal into a discrete time signal. The Nyquist-Shannon sampling theorem states that the sampling frequency must be at least twice the maximum frequency ($f_{\max}$) of the signal to avoid aliasing.

```
load('GroupData_3.mat')
fs = 1 / mean(diff(timevec));
npnts = length(timevec);
```

This calculation, as explained in Haykin and Van Veen [11], ensures accurate representation of the signal's frequency content.

3.2 Time Domain Comparison

A comparison of the original and measurement data in the time domain was performed. The frequency domain of the initial state of the measurement data is displayed.

```
% Determine signal
yOrig = original;
y = measurement;

% power spectrum of signal
yX = abs(fft(y)/npnts).^2;
hz = linspace(0,fs/2,floor(npnts/2)+1);

% plot the data
figure('position', [300, 100, 800, 600]);
subplot(211)
h = plot(timevec,y,'Color','blue'); hold on;
plot(timevec,yOrig,'linew',2,'color','red');
xlabel('time(sec)'), ylabel('power')
title('Time domain','FontSize',14,'FontWeight','bold')
legend({'Measured','Original'})

% plot its power spectrum
subplot(212)
```

```

plot(hz,yX(1:length(hz)),'k','linewidth',1)
xlabel('Frequency (Hz)'), ylabel('Power')
title('Frequency domain of Measurement Signal','FontSize',14,'
      FontWeight','bold')
set(gca,'yscale','log') % log y scale to see more details

```

The power spectrum of the signal was calculated using the Fast Fourier Transform (FFT), following the principles outlined in Oppenheim and Schaffer [1]

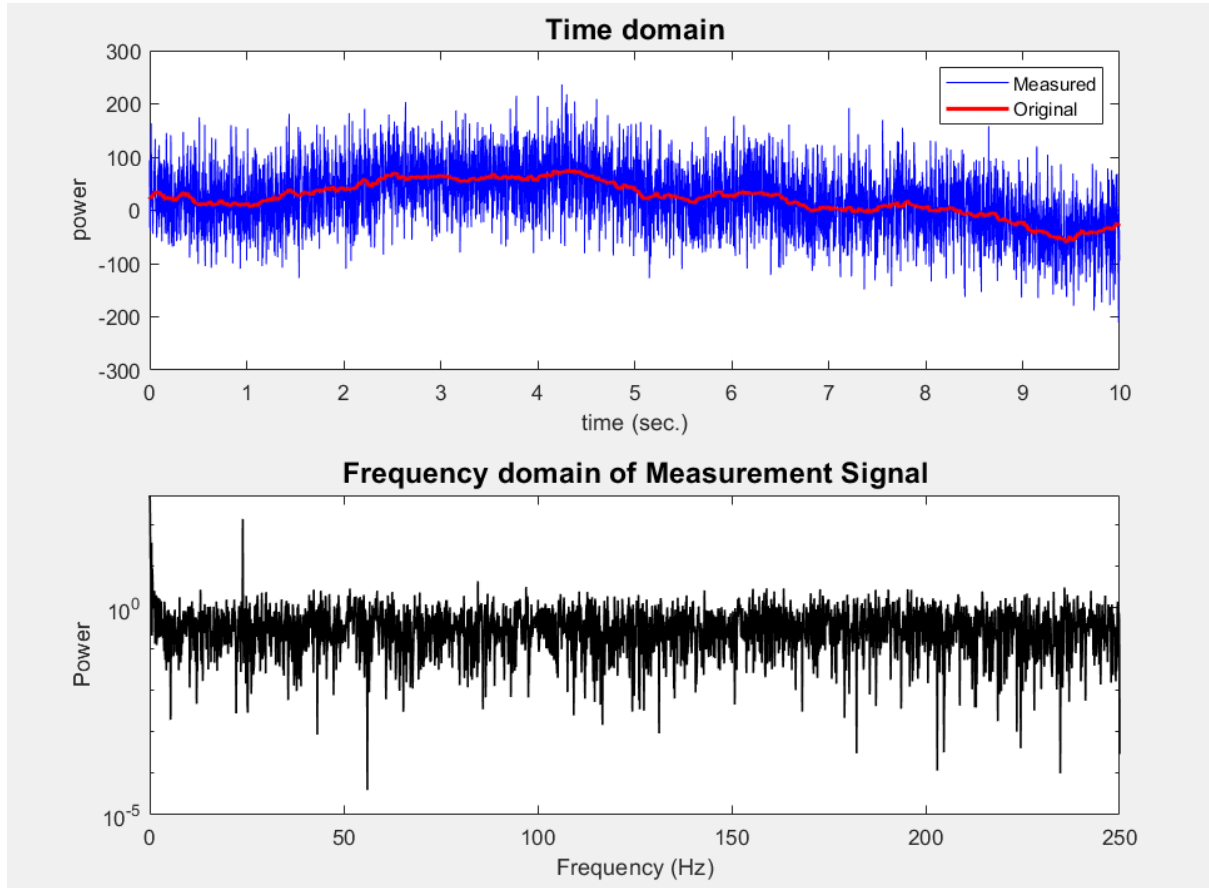


Figure 1: Comparison of the Measured and Original Signals in Time and Frequency Domains

3.3 Frequency Domain of Measurement Signal

The dominant frequency value was examined by throwing the measurement signal into the frequency domain, thus giving an idea about the cut-off value.

```

%for finding fcutoff we need FFT of Measurement signal
...which will be filtered

signalX = fft(y)/npnts;
hz = linspace(0,fs/2,floor(npnts/2)+1);

% amplitude
ampl = abs(signalX);
ampl(2:length(hz)) = 2*ampl(2:length(hz));

```

```

figure('position', [300, 100, 500, 400]);
stem(hz, ampl(1:length(hz)), 'ks-', 'linewidth', 3, 'markersize', 10);

% make plot look a bit nicer
set(gca, 'xlim', [0 fs/2])
xlabel('Frequency (Hz)'), ylabel('Amplitude (a.u.)')
title('Frequency Domain of Original Signal', 'FontSize', 14, 'FontWeight',
      'bold')

```

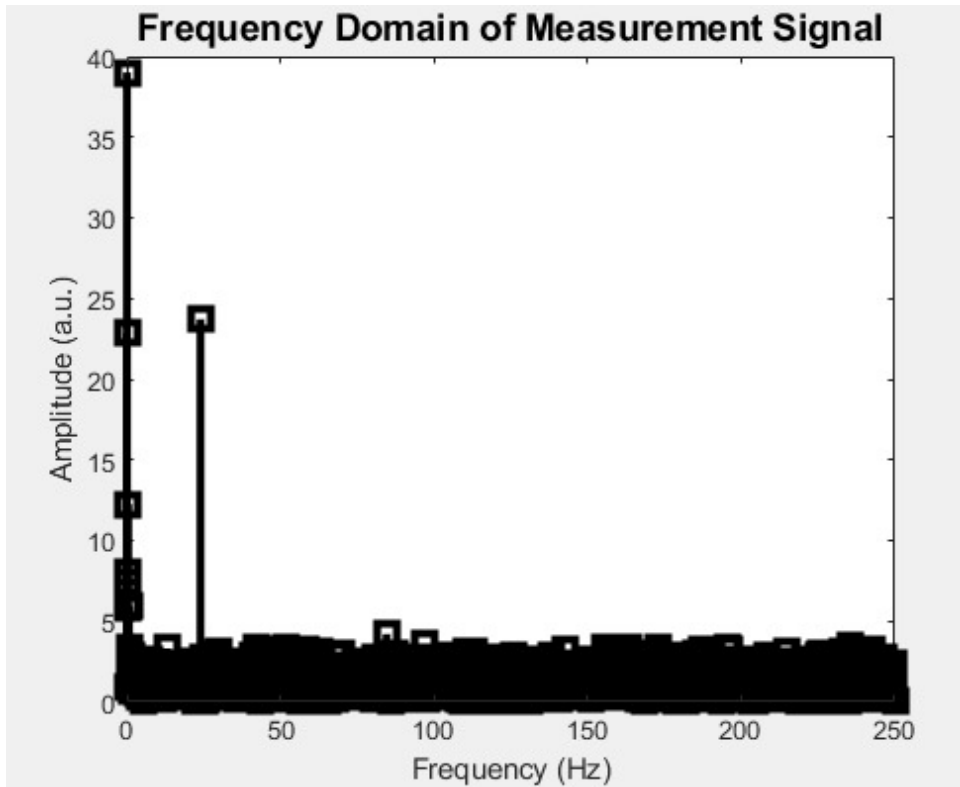


Figure 2: Comparison of the Measured and Original Signals in Time and Frequency Domains

3.4 Generate of Filter Kernel and Power Spectrum and Apply Filter to Data

The filtering process was started under the selected conditions. The filter kernel and the power spectrum of the filter were determined, the ideal shape was created. Filter Kernel, Filter Kernel Spectrum, Measurement, the original state and filtered state of the signal in the time domain, and finally the frequency domain after the filtering process were shown

```

% lowpass filter parameters
fcutoff = 3; % cutoff frequency in Hz
transw = .2; % transition width
order = round(8* fs / fcutoff); % filter order
shape = [1 1 0 0]; % desired frequency response shape
frex = [0 fcutoff fcutoff + fcutoff * transw fs / 2] / (fs / 2);

% create filter kernel using least-squares method
filtkern = firls(order, frex, shape);

```

```

% compute the power spectrum of the filter kernel
filtkernX = abs(fft(filtkern, npnts)).^2;
hz = linspace(0, fs / 2, floor(npnts / 2) + 1);

% plotting
figure('position', [300, 100, 800, 600]);

subplot(321)
plot((-order / 2 : order / 2) / fs, filtkern, 'k', 'linewidth', 3, 'Color',
      'blue')
xlabel('Time (s)')
title('Filter kernel', 'FontSize', 12, 'FontWeight', 'bold')

subplot(322), hold on
plot(frex * fs / 2, shape, 'r', 'linewidth', 1, 'Color', 'blue')
plot(hz, filtkernX(1 : length(hz)), 'k', 'linewidth', 2, 'Color', 'red')
set(gca, 'xlim', [0 60])
xlabel('Frequency (Hz)'), ylabel('Gain')
title('Filter kernel spectrum', 'FontSize', 12, 'FontWeight', 'bold')
legend({'Ideal', 'Actual'})

%% now apply the filter to the data
yFilt = filtfilt(filtkern, 1, y);
subplot(312)
h = plot(timevec, y, 'Color', 'blue'); hold on;
plot(timevec, yFilt, 'linewidth', 2, 'Color', 'red');
legend({'Measurement Signal', 'Filtered'})
xlabel('Time (sec)'), ylabel('Amplitude')
title('Measurement signal vs Filtering Measurement Signal', 'FontSize',
      11, 'FontWeight', 'bold')

%% power spectra of original and filtered signal
yOrigX = abs(fft(y)/npnts).^2;
yFiltX = abs(fft(yFilt)/npnts).^2;

subplot(313)
plot(hz, yOrigX(1:length(hz)), 'Color', 'blue'); hold on;
plot(hz, yFiltX(1:length(hz)), 'Color', 'red', 'linewidth', 2);
set(gca, 'xlim', [0 fs/5], 'yscale', 'log')
legend({'Signal', 'Filtered'})
xlabel('Frequency (Hz)'), ylabel('Power (log)')
hold off

```

The filter kernel was designed using the least-squares method, implemented via the `firls` function, which follows the principles of Parks and Burrus [2].

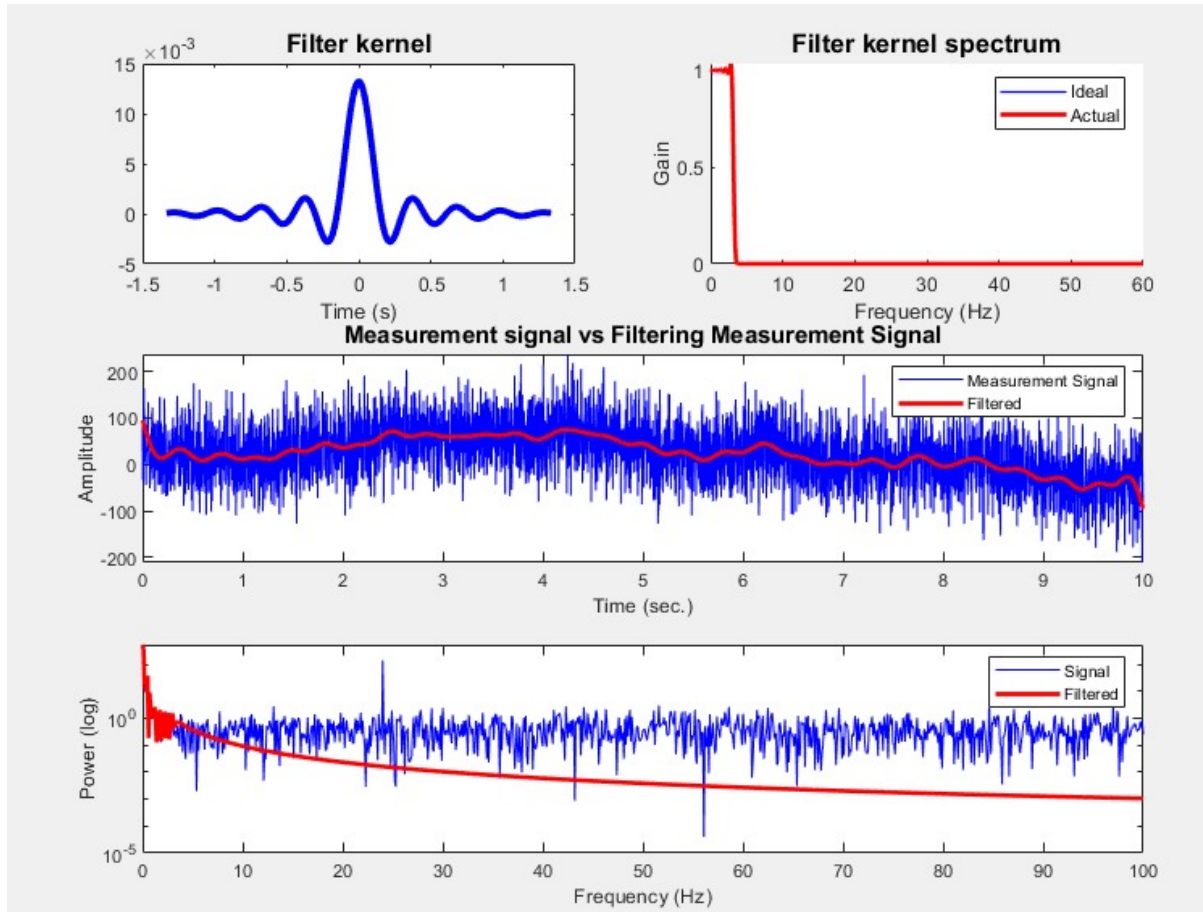


Figure 3: Comparison of the Measured and Original Signals in Time and Frequency Domains

3.5 Original Data Comparison with Filtered New Data

The original data and filtered measurement data were compared in the time domain. Preliminary information about the accuracy of the filter was given.

```
% Obtain the FFT of the filtered signal
yFilt_fft = fft(yFilt);

% Apply inverse Fourier transform to filtered data
yFilt_ifft = ifft(yFilt_fft, 'symmetric');

% Plot to compare original and inverse-transformed filtered data

% Create and position the figure window
figure('position', [300, 100, 600, 450]);
% Plot the original dataset with a thick red line
plot(timevec, yOrig, 'r', 'LineWidth', 2);
hold on;
% Plot the inverse-transformed filtered dataset with blue circles
plot(timevec, real(yFilt_ifft), 'b.', 'LineWidth', 1.5);
xlabel('Time (s)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Amplitude', 'FontSize', 12, 'FontWeight', 'bold');
legend({'Original', 'Inverse-Transformed Filtered'}, 'FontSize', 9);
```

```
title('Comparison of Original and Inverse-Transformed Filtered Data', 'FontSize', 14, 'FontWeight', 'bold');
grid on;
```

The filtered signal was further analyzed using the Fast Fourier Transform (FFT) and its inverse (IFFT), as outlined in Brigham's foundational work on the FFT algorithm [12].

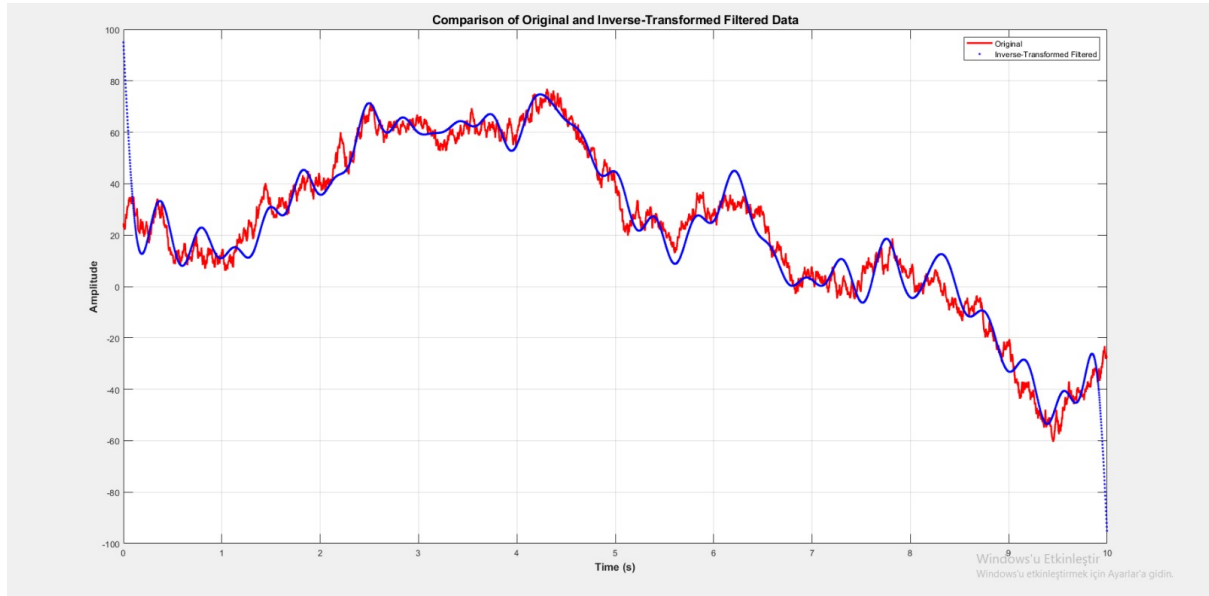


Figure 4: Comparison of The Original Data with Filtered Measurement Signal in Time Domain

4. Results

4.1 Signal Analysis and Parameter Optimization

In the methodology section, detailed explanations of the codes written to avoid confusion are provided. In detail, how the optimal value is determined by assigning different values to key concepts is explained. To ensure clarity and accuracy, detailed explanations of the analysis techniques used are provided, including both time and frequency domain analyses.

4.1.1 Time Domain Analysis

The measured signal and the output of the FIR low-pass filter were compared with the original noise-free signal to assess the filter's effectiveness. The FIR filter was applied using MATLAB's `filtfilt` function, which performs zero-phase filtering by processing the signal in both forward and reverse directions. The filtered signal, computed using MATLAB's `filtfilt` function, exhibited significant noise attenuation while preserving the essential characteristics of the original signal. This bidirectional filtering method prevents phase distortion, ensuring that the signal's temporal structure remains intact. This bidirectional filtering method prevents phase distortion, ensuring that the temporal structure of the signal is not distorted [7].

Visual analysis of the plots demonstrates the alignment between the filtered and original signals in the time domain, highlighting the filter's ability to suppress high-frequency noise without distorting the lower-frequency components. ”

4.1.2 Frequency Domain Analysis

In this section, the frequency characteristics of the measured signal are analyzed to determine the distribution of energy across various frequency components. This graph [Figure 3 bottom] depicts the measured signal's power spectral density (PSD), this demonstrates how much energy is distributed across various frequency components. The x-axis represents frequency in Hz, while the y-axis depicts power on a logarithmic scale. Lower frequencies have higher power, indicating that they are the dominant signal components. At higher frequencies, the power decreases, indicating that these components are primarily due to noise.

The dominant frequency value in the frequency domain was determined with the help of the FFT command. The cut-off frequency (F_{cut}) was determined using FFT, identifying the dominant frequency where energy is highest. This value ensures the removal of high-frequency noise while preserving essential low-frequency components. [7]. F_{cut} was found with this analysis (Figure 2). The dominant frequency value corresponds to the frequency at which the energy is highest. This value is critical for determining the cut-off frequency of the FIR low-pass filter, as it ensures the removal of high-frequency noise while retaining the signal's essential low-frequency components.

4.1.3 Comparison of errors

In this section, the differences between the measured and filtered signals are evaluated to determine the success of the filtering process. The goal is to determine how well the FIR low-pass filter reduces noise while preserving the essential characteristics of the original signal, and to measure the performance of the FIR low-pass filter by evaluating the errors and identifying the underlying sources of these discrepancies.

Measurement errors can be caused by a variety of influences [4, 6]. These include systematic errors (instrumental bias) and random errors (random errors). Systematic errors can be introduced by calibration problems of the measuring instrument or by the design of the instrument and are fixed errors [5]. This error occurs when measuring the same direction. Random errors, on the other hand, are changes in activity rate and will vary from measurement to measurement. This type of noise is usually not part of the original components of the signal. Finally, real errors and inaccuracies occur even if the measurement system does not capture enough. Inadequate speed ratios or cheap devices can cause important details of the signal to be missed, resulting in a mismatch between the measured signal and the original signal.

4.2 How to find the optimum filtering values?

This section describes how to optimise the characteristics used in the design of the FIR low-pass filter. During parameter selection, the goal was to protect the signal's low-frequency performance while effectively suppressing high-frequency noise. To determine the appropriate effect of trial and error methods, comprehensive analysis was used.

4.2.1 How to determine Cut-Off Frequency?

Normally, the cut-off frequency (F_{cut}) equals the dominant frequency, and filtering is applied beyond this value as per the desired filter design [13]. The cut-off frequency is equal to the dominant frequency value and after this value, it can be filtered according to the desired filter. Since the measurement signal has two peaks, normally the second peak, that is, the peak after 24 Hz, should not be exceeded, but since the original data is in the frequency domain, it was seen that this value should be after 0.1 and a value between 0.1-24 Hz was assigned as the appropriate value. The optimum value from these values was determined by looking at the time domain of the original data and the filtered data.

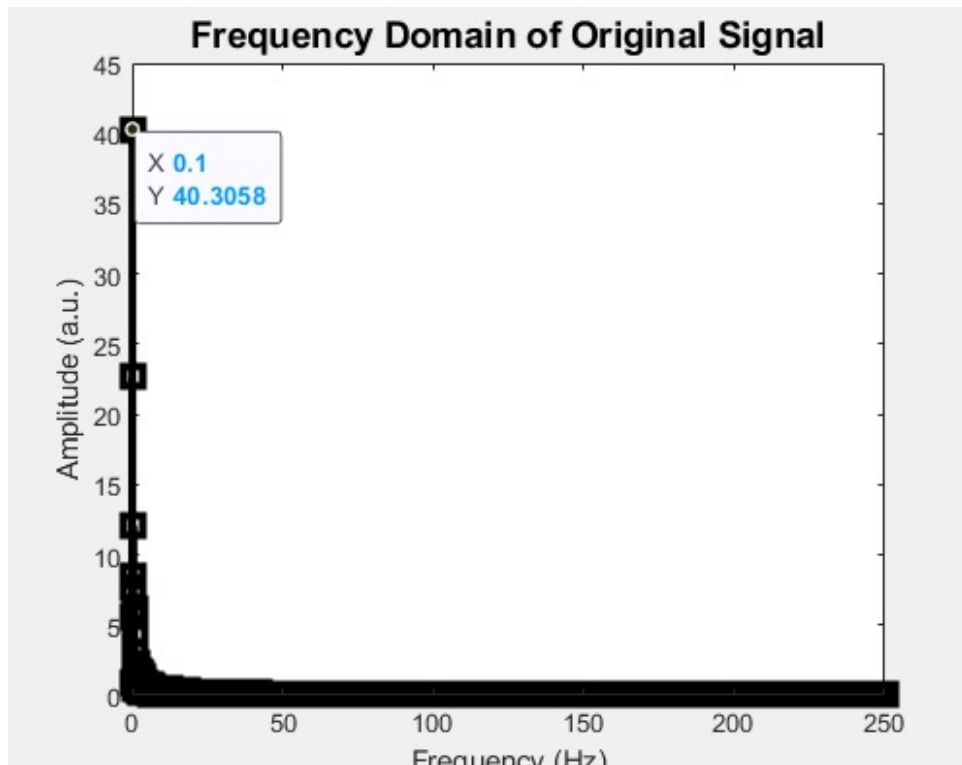


Figure 5: Original Frequency Domain

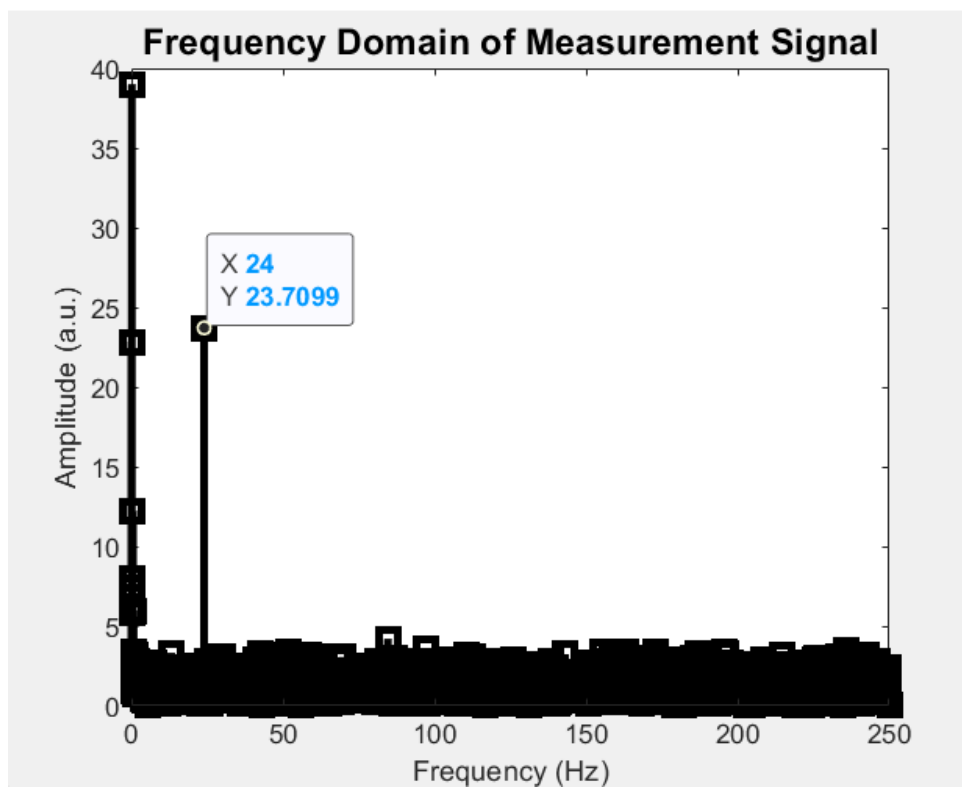


Figure 6: Measurement Frequency Domain

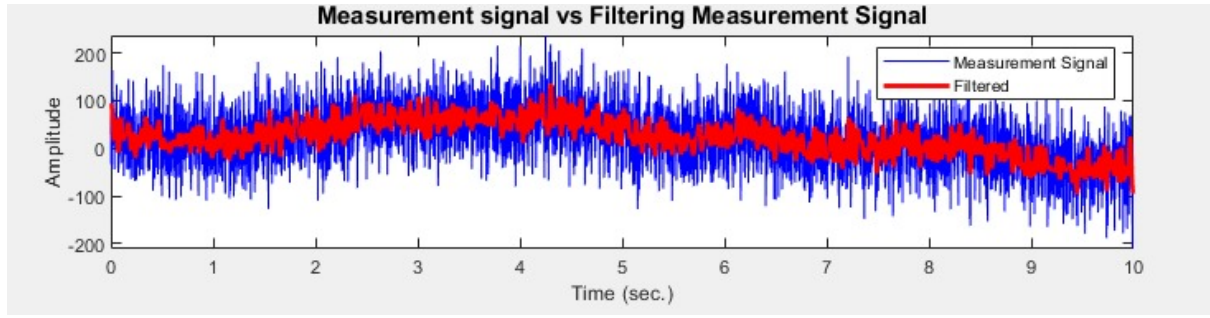


Figure 7: Measurement Signal vs Filtering Measurement Signal Cut-off Frequency at 24 Hz

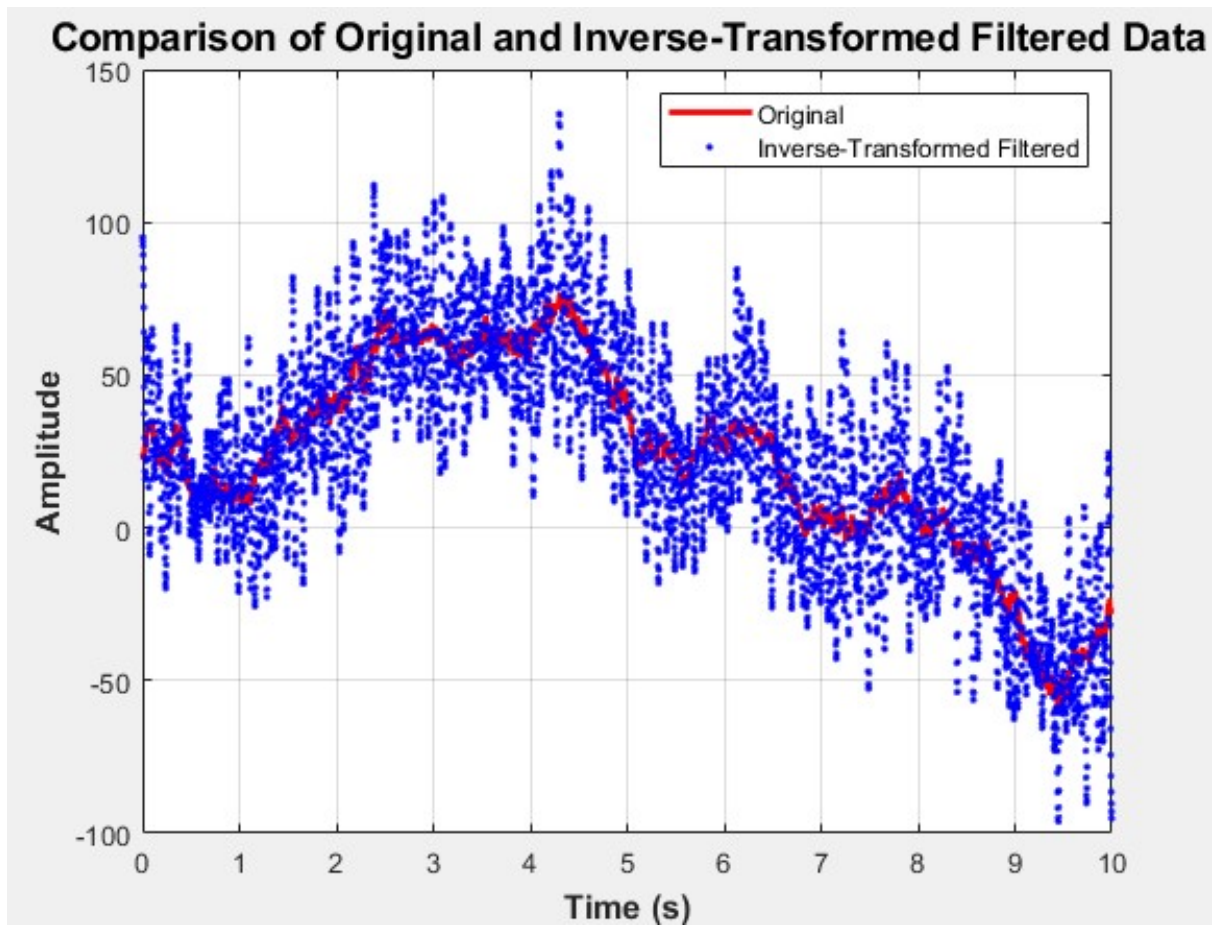


Figure 8: Original Signal vs Filtered Signal Cut-off Frequency 24 Hz

First, the 24 Hz value was selected. As can be seen from the measurement and filtered measurement graphs, the noise data could not be completely deleted and therefore the desired filtered data could not be reached. The desired filtered data was compared with the original data and how much it was scattered was shown.

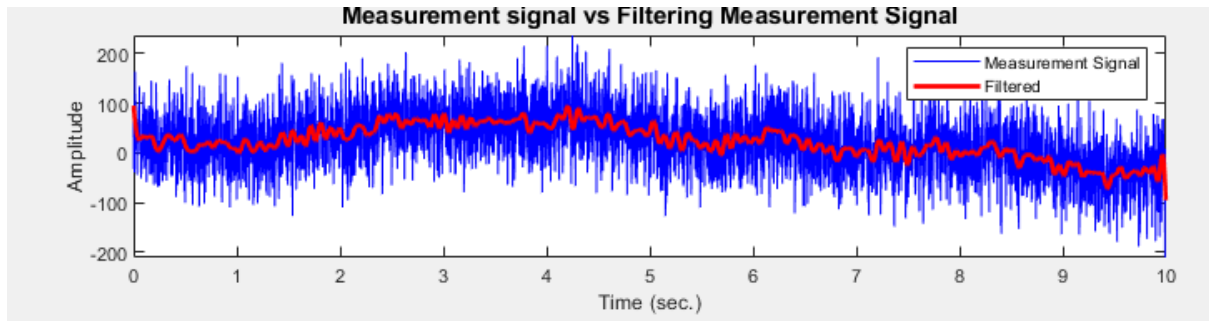


Figure 9: Measurement Signal vs Filtering Measurement Signal Cut-off Frequency at 12 Hz

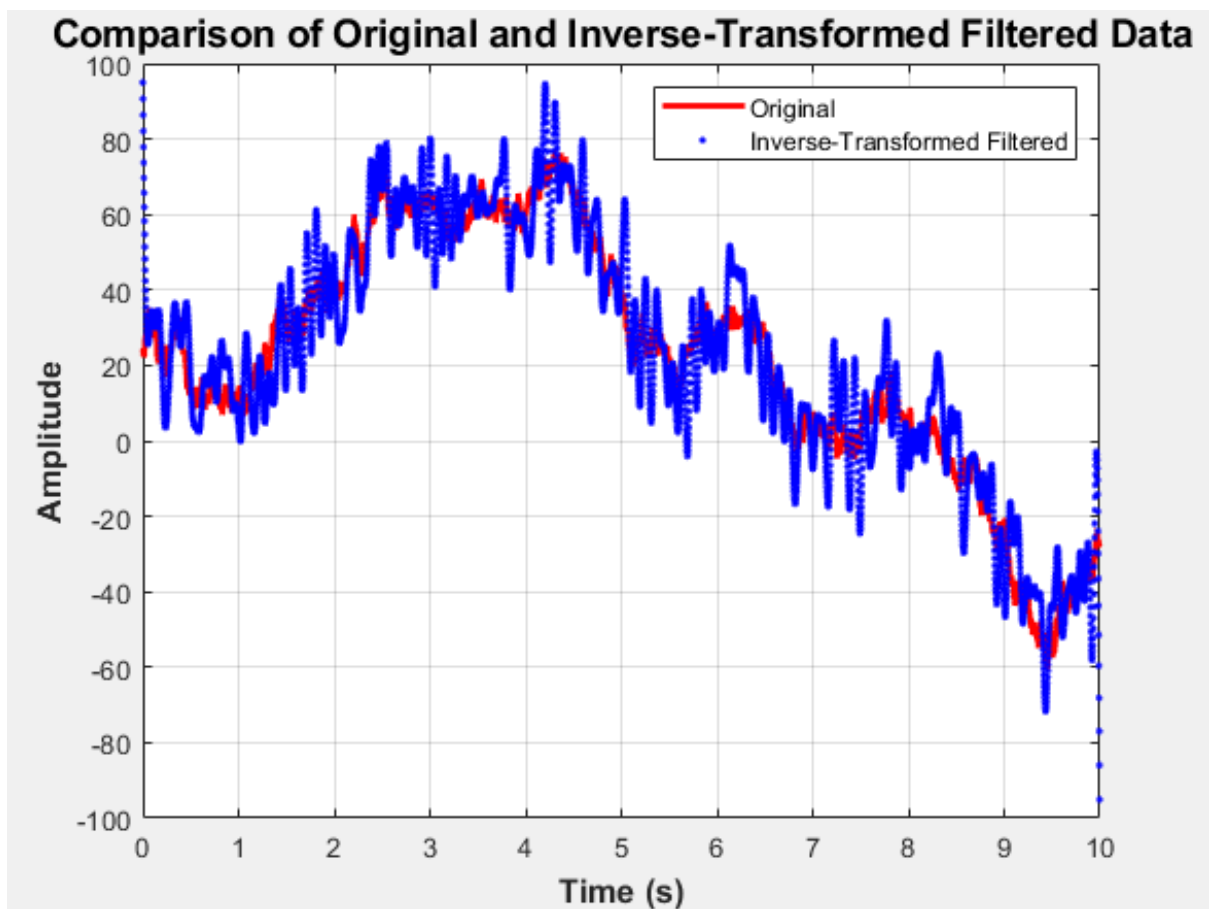


Figure 10: Original Signal vs Filtered Signal Cut-off Frequency 12 Hz

Then, the 12 Hz value was selected and the process was repeated. As a result of the process, it was shown that the noise could not be sufficiently reduced.

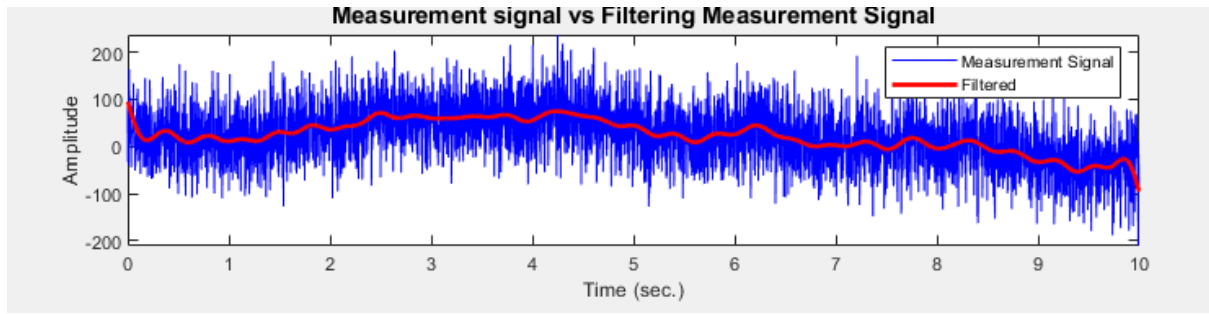


Figure 11: Measurement Signal vs Filtering Measurement Signal Cut-off Frequency at 3 Hz

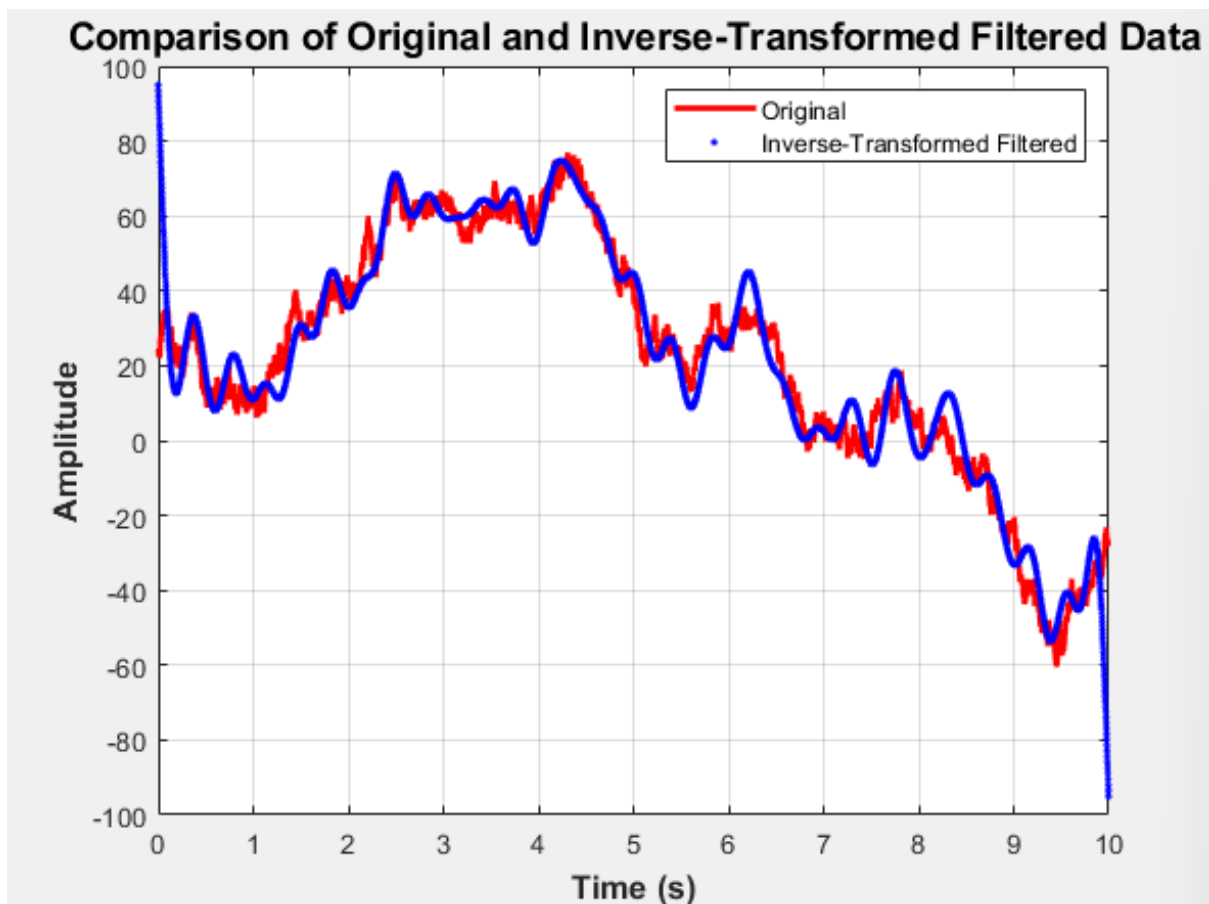


Figure 12: Original Signal vs Filtered Signal Cut-off Frequency 3 Hz

Finally, the process was repeated by cutting from 3 Hz. As a result of the process, it can be seen from the graph under the heading Measurement signal and Filtered Measurement signal that the filtering process is clearly smoother. The filtered data obtained was compared with the original data and was shown to be the best by far.

4.2.2 How to Determine the Order

Optimum values were determined by examining the filter kernel spectrum and finally comparing them with the original data. On the filter kernel spectrum, the values closest to the ideal filter were preferred. This choice increased both the accuracy and efficiency of the filtering process. Filters close to ideal values provide smoother behavior and better fit(Figure14 better than Figure17) on the spectrum. This allowed the selection of optimum parameters that preserve the main structure of the signal while effectively suppressing high-frequency noise.

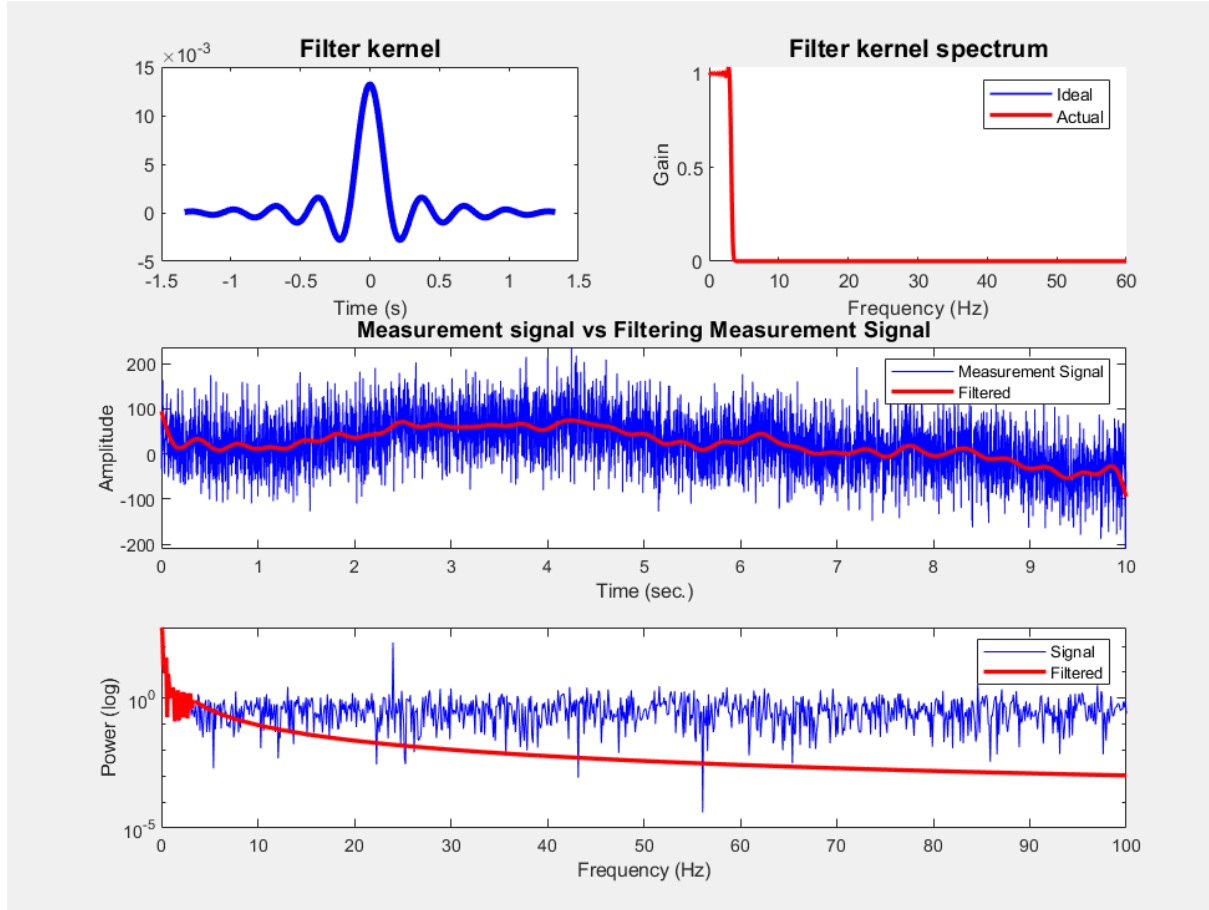


Figure 13: Frequency:3 Width:0,2 Order:8

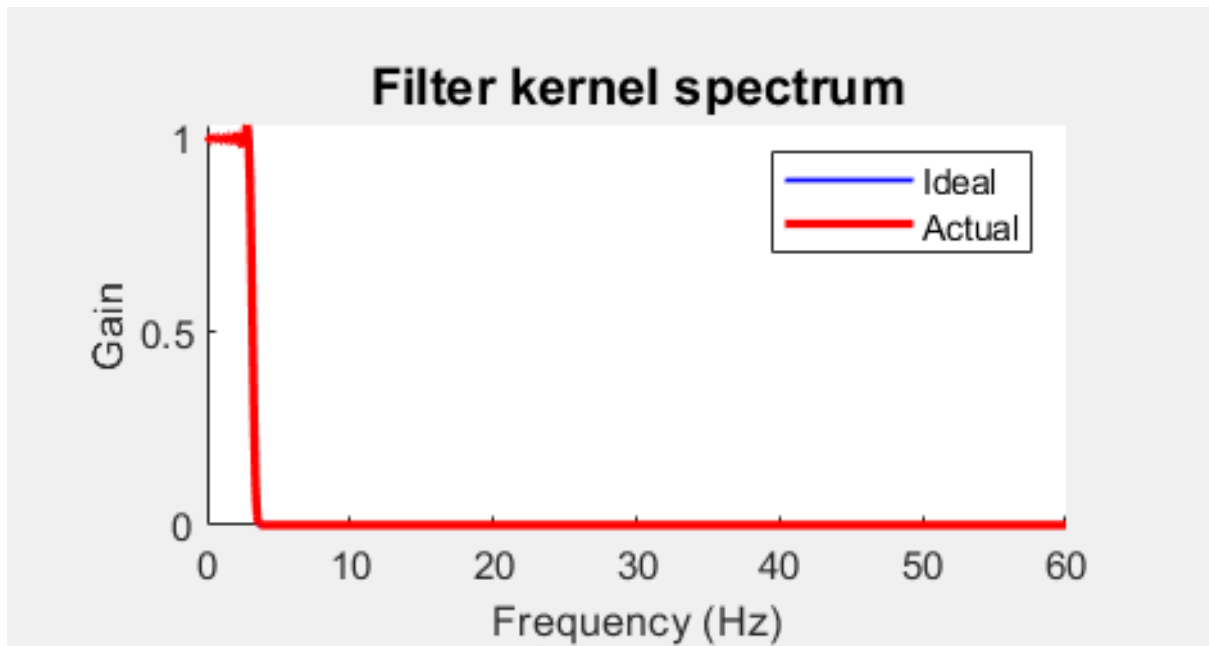


Figure 14: Zoom Filter Kernel Spectrum Frequency:3 Width:0,2 Order:8

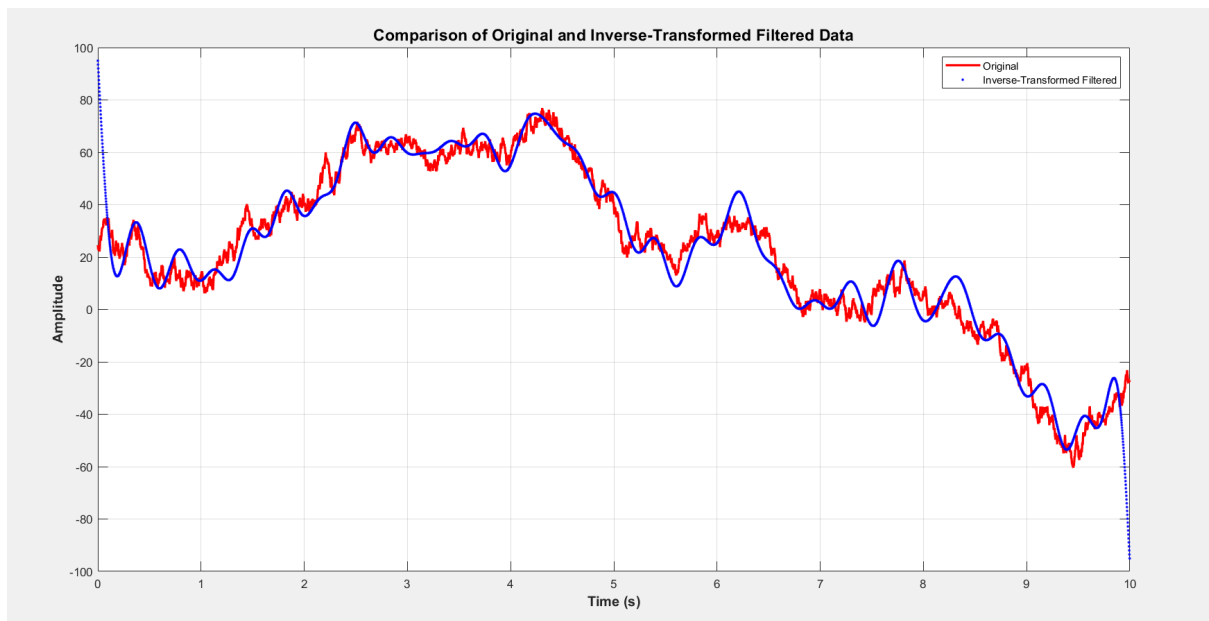


Figure 15: Comparison of Original and Inverse-Filtered Data Frequency:3 Width:0,2 Order:8

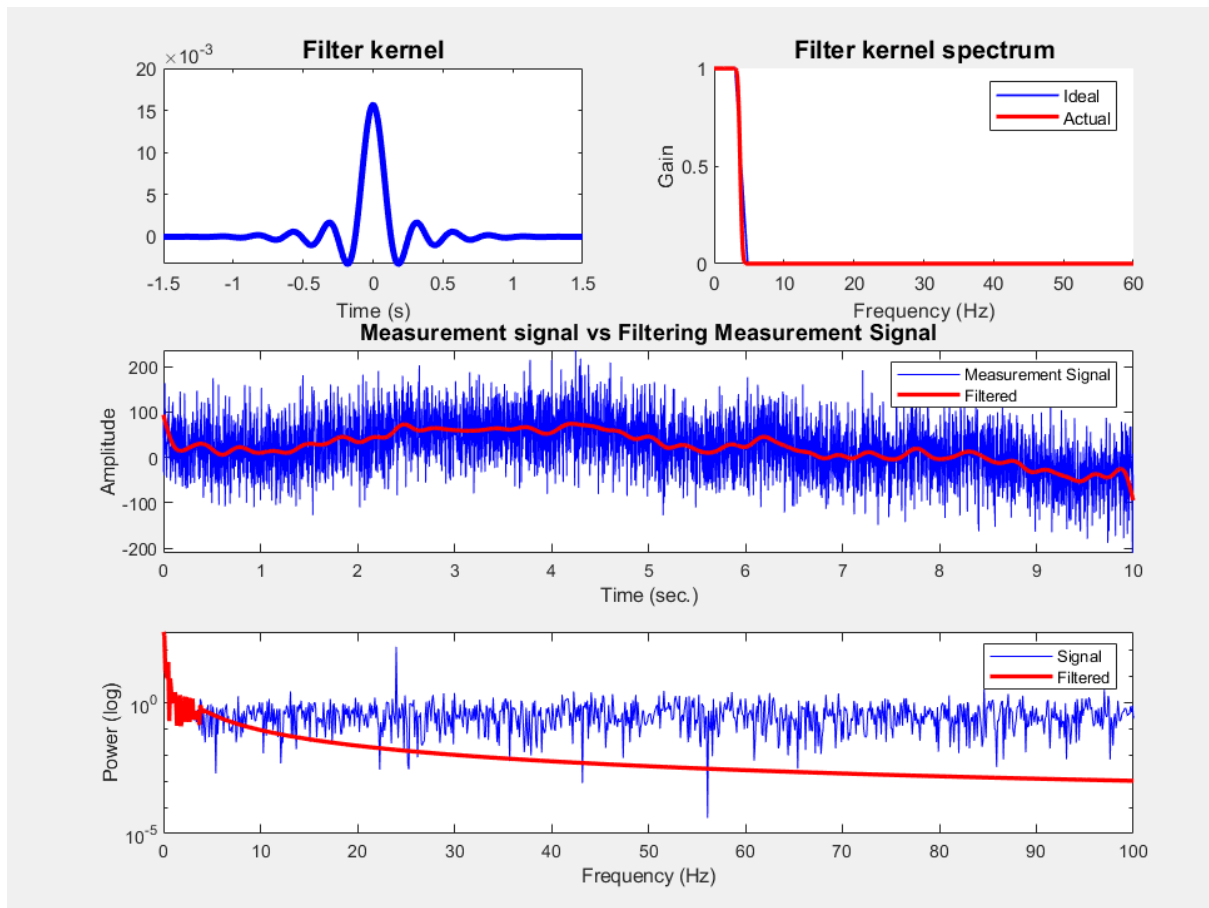


Figure 16: Frequency:3 Width:0,6 Order:9

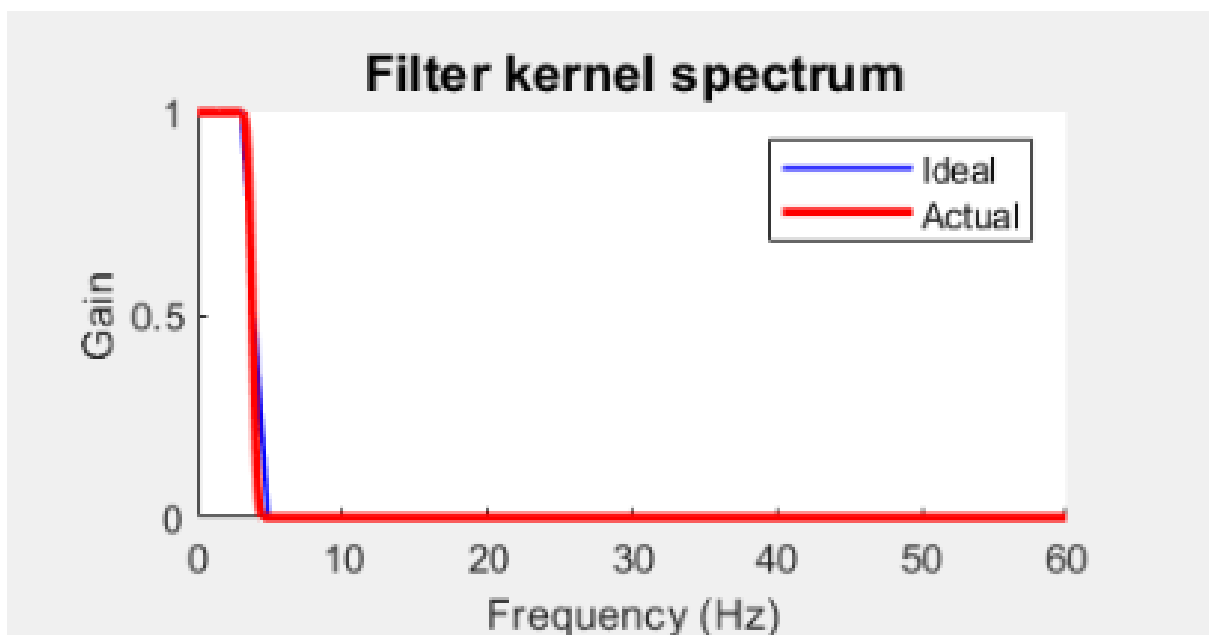


Figure 17: Zoom Filter Kernel Spectrum Frequency:3 Width:0,6 Order:9

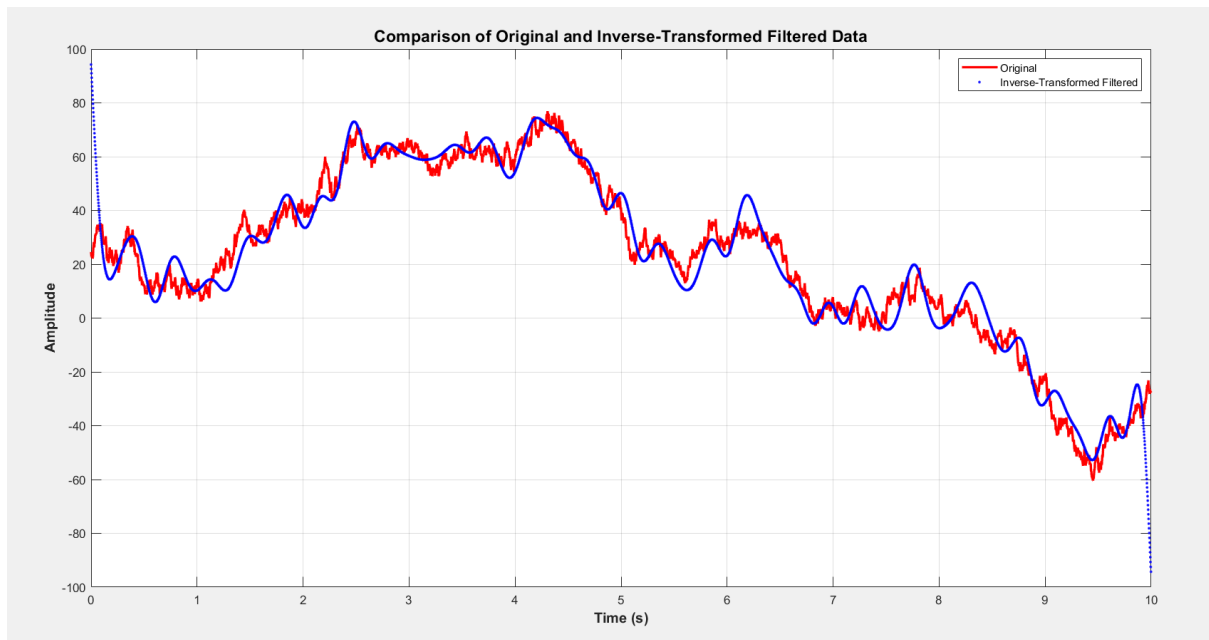


Figure 18: Comparison of Original and Inverse-Filtered Data Frequency:3 Width:0,6 Order:9

5. Discussion

Everything that has been done so far is explained here in more detail, along with the reasons.

1. Introduction: The Original and Measurement data have been carefully examined. It has been concluded that the Measurement data contains noise when compared to the Original data in the time domain. This conclusion was reached by observing the scatter of the measurement data within the time range and analyzing the frequency domain(see Figure1).As described in Shannon’s fundamental work on information theory and noise filtering, noise analysis is critical to signal processing[14].

2. Frequency Analysis: The frequency domain of the measurement data was examined and dominant frequency values of 0.1 Hz and 24 Hz were observed. It was noticed that there was a certain noise between 24 Hz and 0.1 Hz and beyond 24 Hz in these frequency values. [Figure2]

3. 3. Determining the Cut-off Frequency: Initially, it was thought appropriate to perform filtering with a cut-off frequency of 24 Hz. A brainstorming session led to the idea of examining the frequency domain of the original signal. The dominant frequency value of the Original signal was found to be 0.1 Hz. Since a cut-off at 0.1 Hz could not be directly applied, it was anticipated that the cut-off frequency would be between 0.1 Hz and 24 Hz. Since the 0.1 Hz value is dominant, the cut-off frequency is predicted to be close to 0.1 Hz.[Figure2 Figure 5]

4. Filtering and Comparison: To convey these assumptions to the reader, three frequency values between 0.1 Hz and 24 Hz were selected and the resulting filtered signals were compared with the Original signal. Filtering at 3 Hz provided a smoother and more successful result. After determining the cut-off frequency, the filter parameters were optimized through trial and error.[4.2.1]

5. FIR Filtering Method: The FIR filtering method was chosen for this study due to its known stability. MATLAB’s built-in functions `firls` and `filtfilt` were used. An ideal filter shape was defined and the cut-off process was tailored to this shape based on certain transition width and order values. The effects of these processes were shown in detail in previous sections through trial and error. [3.4 Code picture and 4.2.2 section]

6. Conclusion: The filtered signal was transformed from the frequency domain back to the time domain and compared with the Original signal. This comparison clearly indicated that the most suitable cut-off frequency was 3 Hz.(see Fig. 15).

8. References

1. Oppenheim, A. V., and Schafer, R. W., 2010, *Discrete-Time Signal Processing*, 3rd ed., Pearson, Upper Saddle River, NJ.
2. Parks, T. W., and Burrus, C. S., 1972, “Digital Filter Design,” *IEEE Transactions on Audio and Electroacoustics*, 20(2), pp. 195–199. <https://doi.org/10.1109/TAU.1972.1162372>.
3. Lyons, R. G., 2011, *Understanding Digital Signal Processing*, 3rd ed., Pearson, Upper Saddle River, NJ.
4. Bendat, J. S., and Piersol, A. G., 2011, *Random Data: Analysis and Measurement Procedures*, 4th ed., Wiley, Hoboken, NJ.
5. Emery, A. F., and Thompson, W. C., 1989, “Measurement Errors and Their Impact on Experimental Results,” *J. Heat Transfer*, 111(2), pp. 181–187. <https://doi.org/10.1115/1.32500>
6. Coleman, H. W., and Steele, W. G., 1995, “Engineering Application of Experimental Uncertainty Analysis,” *ASME J. Fluids Eng.*, 117(1), pp. 177–185. <https://doi.org/10.1115/1.2>
7. Mitra, S. K., 2011, *Digital Signal Processing: A Computer-Based Approach*, 4th ed., McGraw-Hill, New York.
8. Proakis, J. G., and Manolakis, D. G., 2006, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Prentice Hall, Upper Saddle River, NJ.
9. Ifeakor, E. C., and Jervis, B. W., 2002, *Digital Signal Processing: A Practical Approach*, 2nd ed., Prentice Hall, Upper Saddle River, NJ.
10. Sorensen, H. V., and Jones, D. L., 1987, “A New Approach to Computing the Fast Fourier Transform,” *Proc. IEEE Int. Conf. Acoust., Speech, Signal Process. (ICASSP)*, pp. 436–439. <https://doi.org/10.1109/ICASSP.1987.1169694>.
11. Haykin, S., and Van Veen, B., 2005, *Signals and Systems*, 2nd ed., Wiley, Hoboken, NJ. Smith, S. W., 1997, *The Scientist and Engineer’s Guide to Digital Signal Processing*, 2nd ed., California Technical Publishing, San Diego.)
12. Brigham, E. O., 1988, *The Fast Fourier Transform and Its Applications*, Prentice Hall, Upper Saddle River, NJ.
13. Smith, S. W., 1997, *The Scientist and Engineer’s Guide to Digital Signal Processing*, 2nd ed., California Technical Publishing, San Diego.
14. Shannon, C. E., 1949, “Communication in the Presence of Noise,” *Proceedings of the IRE*, 37(1), pp. 10–21. <https://doi.org/10.1109/JRPROC.1949.232969>.

9. Appendix: MATLAB Code

```
clc
clear
load('GroupData_3.mat')

fs = 1/mean(diff(timevec)); % hz
npnts = length(timevec);

%% Determine signal
yOrig = original;
y = measurement;

% power spectrum of signal
yX = abs(fft(y)/npnts).^2;
hz = linspace(0,fs/2,floor(npnts/2)+1);

% plot the data
figure('position', [300, 100, 800, 600]);
subplot(211)
h = plot(timevec,y,'Color','blue'); hold on;
plot(timevec,yOrig,'linew',2,'color','red');
xlabel('time(sec)'), ylabel('power')
title('Time domain','FontSize',14,'FontWeight','bold')
legend({'Measured','Original'})

% plot its power spectrum
subplot(212)
plot(hz,yX(1:length(hz)),'k','linew',1)
xlabel('Frequency(Hz)'), ylabel('Power')
title('Frequency domain of Measurement Signal','FontSize',14,'FontWeight','bold')
set(gca,'yscale','log') % log y scale to see more details

%%
%for finding fcutoff we need FFT of Measurement signal
...which will be filtered

signalX = fft(y)/npnts;
hz = linspace(0,fs/2,floor(npnts/2)+1);

% amplitude
ampl = abs(signalX);
ampl(2:length(hz)) = 2*ampl(2:length(hz));

figure('position', [300, 100, 500, 400]);
stem(hz,ampl(1:length(hz)),'ks-','linew',3,'markersize',10);

% make plot look a bit nicer
set(gca,'xlim',[0 fs/2])
xlabel('Frequency(Hz)'), ylabel('Amplitude(a.u.)')
title('Frequency Domain of Original Signal','FontSize',14,'FontWeight','bold')
```

```

%%
% lowpass filter parameters
fcutoff = 3; % cutoff frequency in Hz
transw = .2; % transition width
order = round(8* fs / fcutoff); % filter order
shape = [1 1 0 0]; % desired frequency response shape
frex = [0 fcutoff fcutoff + fcutoff * transw fs / 2] / (fs / 2);

% create filter kernel using least-squares method
filtkern = firls(order, frex, shape);

% compute the power spectrum of the filter kernel
filtkernX = abs(fft(filtkern, npnts)).^2;
hz = linspace(0, fs / 2, floor(npnts / 2) + 1);

% plotting
figure('position', [300, 100, 800, 600]);

subplot(321)
plot((-order / 2 : order / 2) / fs, filtkern, 'k', 'linewidth', 3, 'Color',
      'blue')
xlabel('Time(s)')
title('Filter kernel', 'FontSize', 12, 'FontWeight', 'bold')

subplot(322), hold on
plot(frex * fs / 2, shape, 'r', 'linewidth', 1, 'Color', 'blue')
plot(hz, filtkernX(1 : length(hz)), 'k', 'linewidth', 2, 'Color', 'red')
set(gca, 'xlim', [0 60])
xlabel('Frequency(Hz)'), ylabel('Gain')
title('Filter kernel spectrum', 'FontSize', 12, 'FontWeight', 'bold')
legend({'Ideal', 'Actual'})

%%% now apply the filter to the data
yFilt = filtfilt(filtkern, 1, y);
subplot(312)
h = plot(timevec, y, 'Color', 'blue'); hold on;
plot(timevec, yFilt, 'linewidth', 2, 'Color', 'red');
legend({'Measurement_Signal', 'Filtered'})
xlabel('Time(sec)'), ylabel('Amplitude')
title('Measurement signal vs Filtering Measurement_Signal', 'FontSize',
      11, 'FontWeight', 'bold')

%%% power spectra of original and filtered signal
yOrigX = abs(fft(y)/npnts).^2;
yFiltX = abs(fft(yFilt)/npnts).^2;

subplot(313)
plot(hz, yOrigX(1:length(hz)), 'Color', 'blue'); hold on;
plot(hz, yFiltX(1:length(hz)), 'Color', 'red', 'linewidth', 2);
set(gca, 'xlim', [0 fs/5], 'yscale', 'log')
legend({'Signal', 'Filtered'})
xlabel('Frequency(Hz)'), ylabel('Power(log)')
hold off

%%
% Obtain the FFT of the filtered signal
yFilt_fft = fft(yFilt);

```

```

% Apply inverse Fourier transform to filtered data
yFilt_iff = ifft(yFilt_fft, 'symmetric');

% Plot to compare original and inverse-transformed filtered data

% Create and position the figure window
figure('position', [300, 100, 600, 450]);
% Plot the original dataset with a thick red line
plot(timevec, yOrig, 'r', 'LineWidth', 2);
hold on;
% Plot the inverse-transformed filtered dataset with blue circles
plot(timevec, real(yFilt_iff), 'b.', 'LineWidth', 1.5);
xlabel('Time(s)', 'FontSize', 12, 'FontWeight', 'bold');
ylabel('Amplitude', 'FontSize', 12, 'FontWeight', 'bold');
legend({'Original', 'Inverse-Transformed_Filtered'}, 'FontSize', 9);
title('Comparison of Original and Inverse-Transformed Filtered Data', 'FontSize', 14, 'FontWeight', 'bold');
grid on;

```